CONTAINERS AND CLOUD DEVOPS

Mini Project Report

Automating Build, Test, and Deployment Using Git and AWS DevOps Tools

---

This report presents an end-to-end CI/CD implementation for a frequently updated web application using GitHub for version control and AWS services for automated source retrieval, testing, container image build, deployment, and monitoring.

Course	Containers and Cloud DevOps
Project Focus	AWS DevOps services, CI/CD workflow, and Git integration
Deployment Model	Containerized web application on Amazon ECS Fargate
Date	29 March 2026

Problem statement: A development team frequently releases web application updates and needs an automated process to test and deploy every change after code is pushed to the shared repository.

1. Introduction and Objective

Modern software teams require a delivery model where code changes move from development to production quickly, consistently, and with minimal manual intervention. In this mini project, a containerized web application is connected to an AWS CI/CD pipeline so that every push to the main branch triggers a full automation cycle.

The objective is to demonstrate how AWS DevOps services can be combined with Git-based version control to achieve continuous integration and continuous deployment. The implementation covers source integration, automated testing, Docker image build, deployment to the cloud platform, and monitoring of the final release.

The selected implementation uses GitHub, AWS CodePipeline, AWS CodeBuild, Amazon Elastic Container Registry (ECR), Amazon Elastic Container Service (ECS Fargate), CloudWatch, and IAM. This design is highly suitable for a containers and cloud DevOps course because it aligns version control, image-based delivery, and managed deployment.

2. Proposed Architecture

When a developer pushes code to GitHub, AWS CodePipeline receives the update through a secure Git connection. The pipeline then starts a CodeBuild job that installs dependencies, executes automated tests, builds a Docker image, and pushes the new image to Amazon ECR. The deployment stage updates the ECS service so that new containers are launched, traffic is shifted through the load balancer, and application health is observed through CloudWatch.

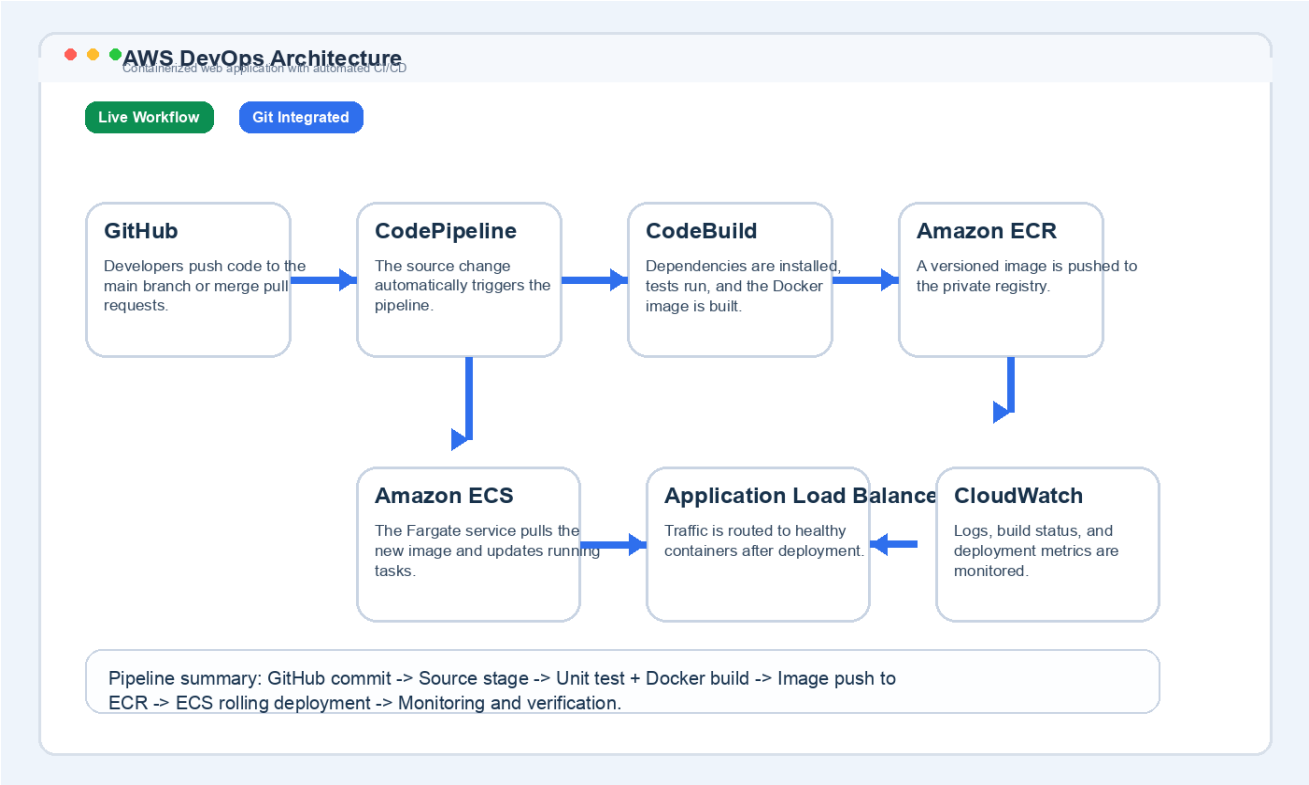

Figure 1. Architecture overview showing GitHub-triggered CI/CD for a containerized application on AWS.

3. AWS DevOps Services Used

Service	Role in the project
GitHub	Stores the application code, branches, pull requests, and full commit history.

AWS CodePipeline	Coordinates the end-to-end workflow from source change to deployment.
AWS CodeBuild	Runs dependency installation, automated tests, Docker image build, and artifact generation
Amazon ECR	Stores versioned container images produced during the build stage.
Amazon ECS Fargate	Runs the web application containers without manual server management.
Amazon CloudWatch	Captures logs, deployment metrics, and runtime health events.
AWS IAM	Provides secure roles and permissions for each stage of the pipeline.

Evaluation point coverage: The services section addresses the AWS DevOps Services requirement by clearly identifying the tools used and explaining their individual contribution in the architecture.

4. CI/CD Pipeline Workflow

1. Source stage: A developer pushes a new commit to the GitHub repository branch connected to the production pipeline.

2. Trigger stage: CodePipeline detects the change using a secure source connection and starts a new execution.

3. Build and test stage: CodeBuild installs packages, runs unit tests, validates the project, and prepares a Docker image.

4. Artifact stage: The image is tagged with the commit identifier and pushed to Amazon ECR for traceable versioning.

5. Deploy stage: CodePipeline updates the ECS task definition and deploys the new image to the running Fargate service.

6. Verification stage: The load balancer health checks the updated tasks and CloudWatch records the deployment outcome.

7. Result: The latest tested application version becomes available to users with minimal manual work and lower release risk.

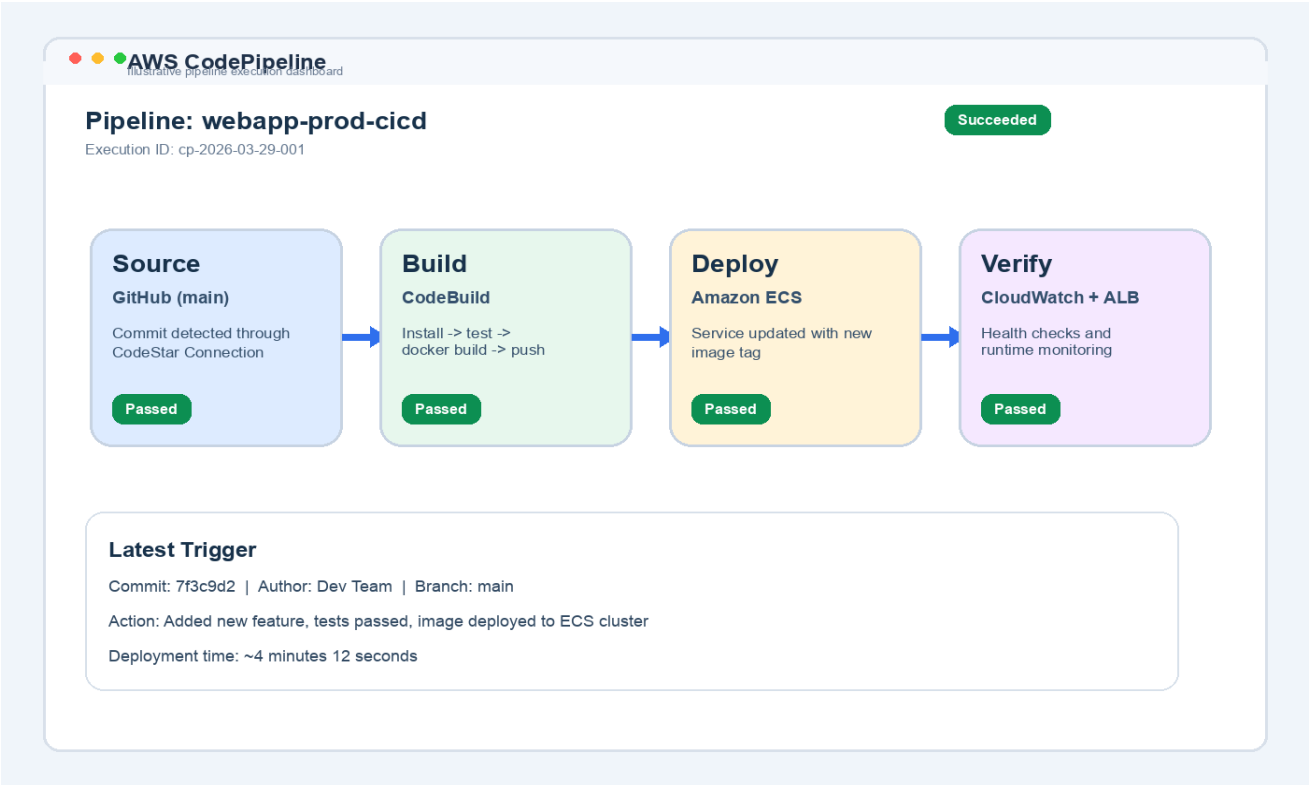

Figure 2. Illustrative CodePipeline view showing the successful completion of source, build, deploy, and verification stages.

5. Integrating the Pipeline with Git

Git provides version control, collaboration, and change tracking for the application source code. The development team works with feature branches for isolated changes, opens pull requests for review, and merges approved updates into the main branch.

The main branch is connected to AWS CodePipeline. As soon as code is pushed or a pull request is merged, the latest commit becomes the input artifact for the pipeline. This ensures that the exact version tested in CodeBuild is the same version deployed.

Git commit hashes are also useful for traceability. The Docker image can be tagged using the resolved source version, allowing the team to identify which release is running in production and quickly roll back to a previous stable image if required.

```
git init
git add .
git commit -m "Initial application version"
git branch -M main
git remote add origin https://github.com/your-org/webapp.git
git push -u origin main
```

The command sequence above shows the basic Git steps to initialize the repository, create the first commit, connect the remote repository, and push the main branch that will feed the AWS pipeline.

6. Implementation Details

Repository setup: A web application repository is created in GitHub with source code, tests, a Dockerfile, and deployment configuration.

Source integration: A CodeStar connection or similar Git integration is configured so AWS can securely read the repository.

Build configuration: The project includes a `buildspec.yml` file that defines install, pre-build, build, and post-build commands.

Container registry: An Amazon ECR repository is created to store the application image after a successful build.

Compute platform: An ECS cluster and Fargate service are created behind an Application Load Balancer.

Pipeline creation: CodePipeline links the source, build, and deploy stages into one automated release flow.

Monitoring: CloudWatch logs and service metrics are enabled to monitor build quality and deployment health.

```
version: 0.2

phases:
  install:
    commands:
      - pip install -r requirements.txt
  pre_build:
    commands:
      - pytest -q
  build:
    commands:
      - docker build -t webapp:$CODEBUILD_RESOLVED_SOURCE_VERSION .
      - docker tag webapp:$CODEBUILD_RESOLVED_SOURCE_VERSION $ECR_REPO_URI:$CODEBUILD_RESOLVED_SOURCE_VERSIO
      - docker push $ECR_REPO_URI:$CODEBUILD_RESOLVED_SOURCE_VERSION
  post_build:
    commands:
      - printf '[{"name":"webapp","imageUri":"%s"}]' $ECR_REPO_URI:$CODEBUILD_RESOLVED_SOURCE_VERSION > imag

artifacts:
  files:
    - imagedefinitions.json
```

7. Testing and Deployment Evidence

The build stage validates the application before any deployment is allowed. If the tests fail, the pipeline stops immediately and production remains unchanged. This practice enforces quality gates and reduces the chance of broken code reaching users.

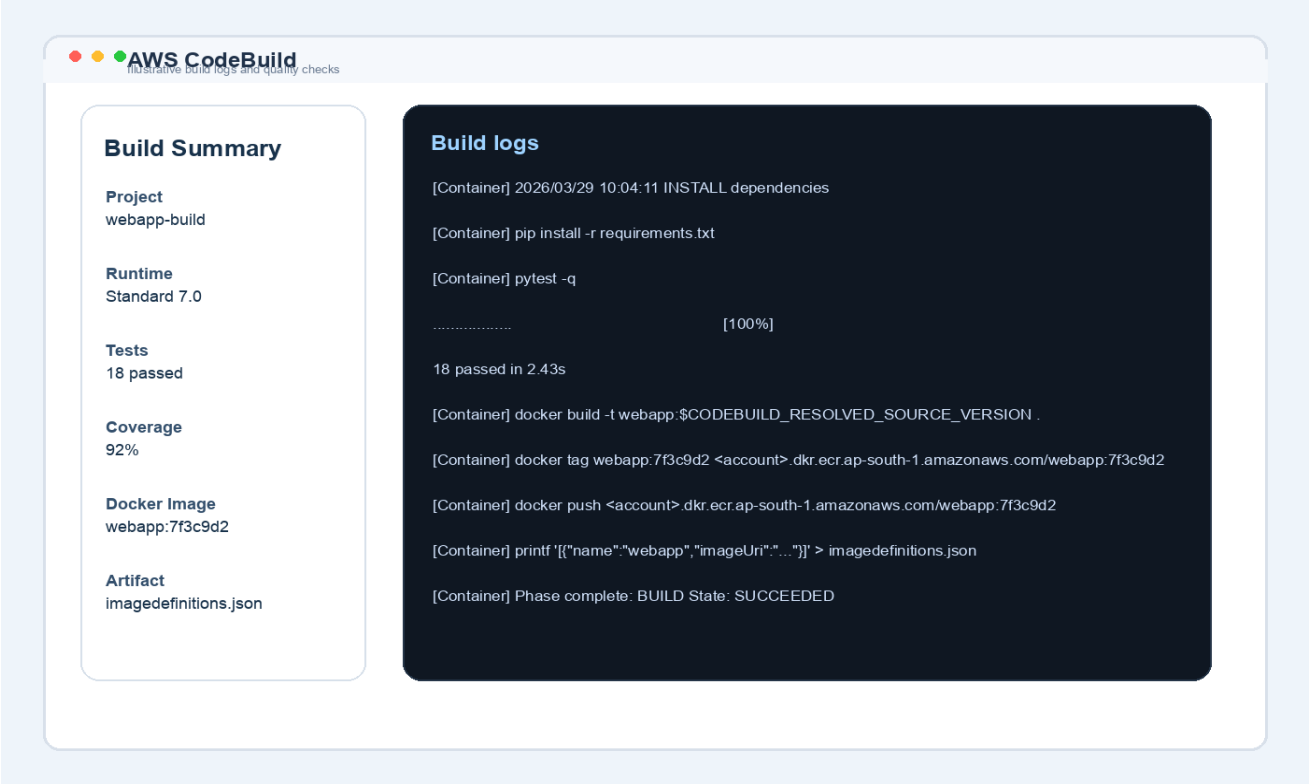

Figure 3. Illustrative CodeBuild output showing dependency installation, automated tests, image creation, and successful artifact generation.

After the build succeeds, the deploy stage updates the running service. Amazon ECS starts new tasks using the latest image, checks health status, and serves traffic through the load balancer only after the new tasks are ready.

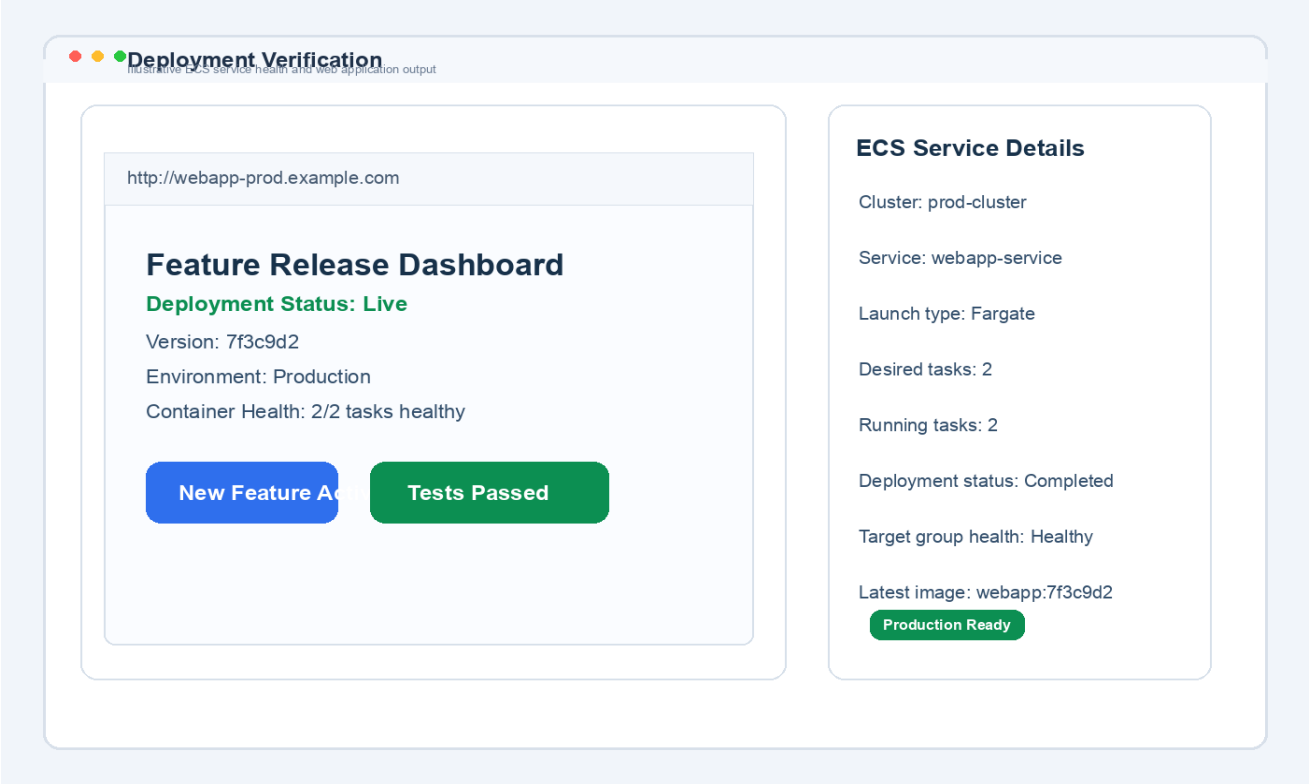

Figure 4. Illustrative deployment verification screen showing the application live with healthy ECS tasks.

8. Advantages of the Solution

Faster releases because every code push is automatically built, tested, and deployed.

Improved consistency because the same pipeline performs every release in a repeatable way.

Better traceability through Git history, pipeline execution logs, and image tags tied to commit IDs.

Higher reliability because automated tests and health checks reduce human error.

Scalability because ECS Fargate can run containers without the team managing virtual machines.

9. Conclusion

This mini project demonstrates a practical cloud-based DevOps implementation for a web application that is frequently updated. By integrating GitHub with AWS CodePipeline, AWS CodeBuild, Amazon ECR, and Amazon ECS, the development team can automate source retrieval, testing, image packaging, deployment, and verification. The resulting CI/CD process is faster, safer, and more maintainable than manual release steps, and it directly satisfies the assignment requirement to automate build, test, and deploy using Git and AWS DevOps services.

10. Submission Note

The figures in this report are polished illustrative screenshots created to document the expected AWS workflow in an offline environment. If your faculty requires live console evidence, replace these figures with screenshots captured from your own AWS account while keeping the same report structure.